

Module 1 — LLM Foundations

This is the LLM-foundations module of a knowledge base on production enterprise AI; it assumes no ML background and can be understood on its own.

Why this module exists. Every design decision in later modules — retrieval, tool calling, agents, evaluation, cost management — is downstream of how a language model actually works. If you understand tokenization, embeddings, attention, context windows, and why models hallucinate, the rest of the series stops being magic and becomes engineering. This module is the physics; everything after is the architecture built on top of it.

This module covers what an LLM fundamentally is, tokenization, embeddings and vector spaces, the transformer architecture, attention, context windows, the training stages, inference parameters such as temperature and sampling and logprobs, and finally why hallucinations occur.

What an LLM fundamentally is

Why it matters. A Large Language Model is, at its core, a next-token predictor: given a sequence of text, it outputs a probability distribution over what the next chunk of text is likely to be. That's it. It is not a database, not a search engine, not a reasoning oracle with a fact store. Every impressive behavior — writing code, drafting a contract clause, planning a multi-step task — is an emergent consequence of doing next-token prediction extraordinarily well over a vast amount of text.

This single fact explains most of the enterprise architecture that follows. Because the model reasons fluently but knows unreliably, we bolt on retrieval, structured knowledge, and tools to supply trustworthy, current, permissioned facts and the ability to act on the world.

Analogy. Think of an LLM as an extraordinarily well-read improv actor. It has absorbed the patterns of language, code, and argument from reading nearly everything, and it can extend any prompt in a plausible, stylistically appropriate way. But ask it for a specific customer's contract renewal date and it will produce something that sounds exactly like a renewal date — whether or not it has ever seen the real one. The actor's job is fluency, not truth. Our job as system builders is to feed it the script, meaning the facts, and give it a phone, meaning the tools.

How it works. The model is a large neural network with billions of parameters that maps an input sequence of tokens to a probability distribution over the next token. Generation is autoregressive: predict one token, append it, feed the whole thing back in, predict the next, and repeat until a stop condition. There is no separate lookup step — knowledge is smeared across the weights as statistical regularities learned during training.

Where it goes wrong. The single most expensive enterprise misconception is treating the model as a knowledge base. It is a reasoning-and-language engine. Knowledge must be supplied or verified externally.

Tokenization

Why it matters. Models don't read characters or words — they read tokens, sub-word units produced by a tokenizer. Tokenization is the process of chopping text into these units and mapping each to an integer ID. It matters for three practical reasons: cost, because you're billed per token; context limits, because windows are measured in tokens; and behavior, because weird tokenization causes weird failures, for example on numbers, code, and non-English text.

How it works. The dominant scheme is Byte-Pair Encoding, or BPE, and its variants such as WordPiece, SentencePiece, and tiktoken-style byte-level BPE. BPE starts from characters or bytes and iteratively merges the most frequent adjacent pairs into a fixed vocabulary, commonly fifty thousand to two hundred thousand or more tokens. Common words become single tokens, such as "the" or "customer"; rare words split into pieces, so "Salesforce" might become "Sales" plus "force"; arbitrary strings fall back to bytes.

Rules of thumb for English: roughly four characters per token, or roughly zero point seven five words per token — so about one thousand tokens is roughly seven hundred and fifty words. But treat this as a tokenizer-specific, drifting heuristic, not a law: it degrades for code, JSON, non-Latin scripts, and long numbers, and it changes when a vendor changes tokenizers. Concrete example: Anthropic's tokenizer introduced with Claude Opus 4.7, and later models, produces roughly 30% more tokens for the same text than earlier Claude models — enough to blow a word-count-derived budget estimate.

The main options. Each model family has its own tokenizer; token counts are not interchangeable across vendors — and not even across generations from the same vendor, so recount whenever the tokenizer changes, as in the Opus 4.7 example above. Use the vendor's tokenizer or counter to estimate cost and fit — for example, OpenAI's tiktoken, Anthropic's token-counting endpoint, or HuggingFace tokenizers.

When to use it (and when not to). When estimating cost or truncation risk, always count with the target model's tokenizer, not a generic word count. When designing prompts that include tabular or numeric data, remember it tokenizes expensively.

Where it goes wrong. Arithmetic and digit handling are historically weak partly because numbers tokenize awkwardly — a long number can split into odd fragments. Prefer tool-calling to a calculator for anything that must be exact.

Non-English content costs more tokens and can degrade quality.

Watch out for silent truncation: if input exceeds the window, middle content may be dropped or the request rejected. Always budget tokens explicitly.

A concrete example. A CPQ assistant that pastes a forty-page master services agreement into the prompt may find the PDF is sixty thousand tokens — blowing the budget and the cost model. This is exactly the pressure that pushes you toward retrieval: fetch only the relevant clauses instead of the whole document.

Embeddings and vector spaces

Why it matters. An embedding is a list of numbers, a vector — for example 768 or 1,536 or 3,072 dimensions — that represents the meaning of a piece of text as a point in high-dimensional space. The key property: semantically similar things land near each other. "Cancel my subscription" and "I want to end my plan" produce nearby vectors even with no shared words. Embeddings are the mathematical foundation of semantic search and therefore of retrieval.

Analogy. Imagine a vast library where books are shelved not by title but by meaning — cookbooks near each other, noir novels in another region, tax code in another. An embedding is the shelf coordinate. To find relevant material you don't match keywords; you go to the right neighborhood.

How it works. An embedding model, often a transformer encoder, maps text to a vector such that cosine similarity — the angle between vectors — or dot product correlates with semantic similarity. So how does text come to have meaningful coordinates? The model is trained contrastively: it is shown millions of related pairs, such as a question and its answer, or a sentence and its paraphrase, along with unrelated pairs, and tuned to pull related pairs close together and push unrelated ones apart in the vector space. Do that at scale and the geometry ends up encoding meaning. This is also why you must embed queries and documents with the same model — they must share one geometry — and why some models use different "query" versus "passage" encodings. The famous, if oversimplified, illustration: the vector for king, minus man, plus woman, lands near queen — relationships become directions in the space.

Two distinct notions of "embedding" appear in this series. First, token embeddings, which are internal to the model; each token maps to a vector the transformer processes. Second, sentence or document embeddings, which are a single vector for a whole passage, produced by a dedicated embedding model and used for retrieval.

The main options. As of mid-2026 the leaders have shifted: Google Gemini Embedding and open-weight Qwen3-Embedding top MTEB, with Cohere Embed v4, which is multimodal and long-context, and Voyage-3.5 strong for domain retrieval; OpenAI's text-embedding-3 family and open models such as bge, e5, nomic, gte, and Jina remain widely used. The open-versus-proprietary quality gap has largely closed. They differ in dimensionality, max input length, multilingual and multimodal quality, domain fit, and cost. Some support

Matryoshka embeddings, which have truncatable dimensions for a size-versus-quality tradeoff. Rankings drift monthly — verify on the current MTEB leaderboard.

When to use it (and when not to). Choose an embedding model on: retrieval quality on your data, which you should measure; dimensionality, which drives storage and latency cost; max input length, which affects chunking; multilingual needs; and whether you can self-host for data-residency reasons. Do not assume the biggest model is best for retrieval; domain-tuned smaller models often win.

Where it goes wrong. Model mismatch: you must embed queries and documents with the same model; mixing embedding models corrupts the geometry.

Re-embedding cost: changing embedding models means re-indexing your whole corpus.

Embeddings capture semantics, not truth or recency — two false statements about the same topic sit close together.

A concrete example. A support agent bot embeds every knowledge-base article and every incoming ticket. A ticket saying "the widget won't sync after the update" retrieves the article "Resolving synchronization failures following firmware upgrades" — no shared keywords, but neighbors in vector space.

The transformer architecture

Why it matters. The transformer, from Vaswani and colleagues in the 2017 paper *Attention Is All You Need*, is the neural network architecture behind virtually all modern LLMs. Its breakthrough was replacing sequential recurrence with attention, which lets the model process all positions in parallel and directly relate any token to any other. This is why models could scale to hundreds of billions of parameters and long contexts — parallelism is trainable on modern hardware.

How it works — the pipeline. First, tokenize the input into integer IDs.

Second, embed each token into a vector, and add positional information so the model knows word order — modern models use rotary or learned relative position encodings.

Third, pass through a stack of transformer blocks, from dozens to more than a hundred. Each block does two complementary things. One is a multi-head self-attention layer — the mixing or routing step: each token pulls in information from other tokens to build context. The other is a feed-forward network, or FFN — a per-token nonlinear transformation that is, loosely, where much of the model's learned knowledge and pattern-application lives. Attention decides what to combine; the FFN decides what to do with it. This is the mechanical hook for the earlier idea that knowledge is smeared across the weights — a lot of it sits in these FFN layers. Each block also uses residual connections — a bypass lane that adds each layer's output back to its input, so a layer only has to learn a small correction rather than rebuild the whole signal, which is what

makes hundred-layer networks trainable — and layer normalization, which rescales activations to keep them numerically stable.

Fourth, a final layer projects the top representation to a distribution over the vocabulary, giving the next-token probabilities.

So what is the model actually computing across those layers? Roughly: early layers resolve surface features such as syntax and references, middle layers build higher-level meaning and relationships, and later layers assemble the prediction — each layer refining the representation a little, the residual small-correction idea, until the final layer reads out the next-token distribution.

Decoder-only versus encoder-only versus encoder-decoder. Most generative LLMs, including GPT, Claude, Llama, and Gemini, are decoder-only — they predict the next token given all previous tokens. Encoder-only models, in the BERT style, are used for embeddings and classification. Encoder-decoder models, such as T5 and the original translation transformers, map an input sequence to an output sequence.

Variants worth knowing. Mixture-of-Experts, or MoE, models activate only a subset of expert sub-networks per token, giving large capacity at lower inference cost; several frontier models are believed to use MoE, though labs rarely disclose architectures. State-space and hybrid models — Mamba-3 and SSM-transformer hybrids like Jamba and Nemotron-H — have reached production for throughput- and long-context-sensitive workloads, trading some peak quality for cheaper long sequences, though transformers still dominate the frontier.

Where it goes wrong. Architecture is rarely something an enterprise consumer tunes — you consume models via API. The relevant leverage is knowing that attention cost historically grows with the square of the sequence length, which is why long contexts are expensive and why retrieval beats stuffing everything in.

A concrete example. You don't build transformers; you choose between them — a small fast model for classification versus a large model for complex reasoning — and you route between them. Understanding the architecture is what lets you reason about that cost-versus-quality frontier.

Attention

Why it matters. Attention is the mechanism by which, when processing a given token, the model decides which other tokens matter and how much. It's how the model handles context, resolves references — for example, which noun does "it" refer to? — and connects distant but related information. Attention is the single most important idea in the transformer.

Analogy. Reading the sentence "The quote expired because it was never countersigned," your brain instantly binds "it" to "quote," not "because." Attention is the model doing that binding — for every token, against every other token, at every layer, simultaneously.

How it works. For each token, the model computes three vectors: a Query, meaning what am I looking for; a Key, meaning what do I offer; and a Value, meaning what do I contribute if attended to. The attention score between two tokens is the dot product of one's Query with another's Key; scores are normalized with softmax into weights; each token's new representation is the weighted sum of all Values. Multi-head attention runs several of these in parallel, each learning different relationship types such as syntax, coreference, and topic, then combines them.

Self-attention relates tokens within one sequence. Causal, or masked, attention in decoder models prevents a token from seeing future tokens, which is essential for next-token prediction.

The cost problem. Naive attention compares every token to every other token, so cost grows with the square of the sequence length. This is the root of why long context is expensive. Engineering responses include FlashAttention, which is memory-efficient exact attention; KV-caching, which reuses computed Keys and Values across generation steps — critical for inference speed and the basis of prompt caching; sparse or sliding-window attention; and grouped-query attention.

Where it goes wrong — "lost in the middle." Empirically, models attend most reliably to the beginning and end of a long context and can neglect material buried in the middle. Honest framing: the effect is robustly replicated across many models and long-context benchmarks; the precise mechanism is still debated, whether positional-encoding biases or primacy and recency patterns in training data — so treat it as an empirical constraint to engineer around, not a settled law. The practical consequence: placing critical instructions and data at the edges, and retrieving less but more relevant context, beats dumping everything in.

A concrete example. When an agent reads a thirty-page contract to answer "what's the termination notice period?", attention is what lets it connect the question to the one relevant clause. But if that clause sits in the middle of a giant stuffed context alongside twenty irrelevant documents, "lost in the middle" can cause it to miss — an argument for precise retrieval and reranking.

Context windows

Why it matters. The context window is the maximum number of tokens the model can consider at once — the prompt plus the generated output must fit. It is the model's working memory. It has grown from around two thousand tokens in 2020 to one million tokens now being standard across frontier hosted models, including recent Claude, Gemini, and others, with some open-weight models such as Llama 4 Scout

advertising up to ten million — though, as below, effective recall degrades well before the advertised limit. Context size dictates how much instruction, retrieved data, conversation history, and tool output you can supply — and it caps how big a document you can process in one shot.

Analogy. The context window is the model's desk. Anything on the desk it can use right now; anything filed away it cannot see unless you fetch it and put it on the desk. Retrieval is the process of fetching the right files onto a finite desk.

How it works, and the caveats. The window covers everything: the system prompt, tool definitions, history, retrieved documents, and the model's own output.

Bigger isn't free. Cost and latency scale with tokens processed; long contexts are slower and pricier.

Effective context is less than advertised context. A model may accept one million tokens but use them unevenly — the "lost in the middle" effect and degradation on needle-in-a-haystack-with-distractors tasks mean that stuffing the window is not the same as reasoning well over it. Long context is not a substitute for good retrieval — now repeatedly confirmed on million-token models by long-context benchmarks such as RULER, LongBench v2, and HELMET: larger windows shift the degradation further out but do not remove it.

When to use it (and when not to) — long context versus retrieval. Use long context when the whole relevant corpus is small enough to fit, changes constantly, and precision of retrieval is hard — for example, "summarize this one two-hundred-page deposition."

Use retrieval when the corpus is large, must be permissioned or filtered, changes per-user, or when the cost and latency of stuffing is prohibitive, which covers almost all enterprise knowledge bases. Often you combine them: retrieve a focused set, then use a long window to reason over it.

Where it goes wrong. Blowing the budget with verbose tool outputs or long histories; assuming "one million context" means "reads a million tokens well"; forgetting that output tokens count against the window.

A concrete example. A deal-desk agent could theoretically load a customer's entire Salesforce history into a long context, but it's cheaper, safer through entitlement filtering, and more accurate to retrieve the last three opportunities and the active quote.

The training stages: pretraining, fine-tuning, instruction-tuning, alignment

Why it matters. A production model is built in stages, and knowing which stage does what tells you which lever to pull when the model isn't behaving: do you need better prompts, retrieval, or actual fine-tuning?

This is the conceptual map; the mechanics of fine-tuning come later.

How it works — the stages. First, pretraining. The model learns language and world patterns by predicting the next token over a massive, diverse corpus of web, books, and code. This is where the bulk of knowledge and capability comes from. It's enormously expensive and done by frontier labs, not enterprises. The output is a base model — fluent but not helpful; it just continues text.

Second, supervised fine-tuning, or SFT, also called instruction-tuning. The base model is trained on curated instruction-and-good-response pairs so it learns to follow instructions and answer rather than merely continue. This turns a text-continuation engine into an assistant.

Third, alignment, or preference optimization. Techniques tune the model toward responses humans prefer — helpful, harmless, honest, correctly formatted, and appropriately cautious. Briefly, how each works. RLHF, Reinforcement Learning from Human Feedback: humans rank competing model outputs, a reward model is trained to mimic those rankings, and the LLM is then optimized to score well against that reward model. DPO, Direct Preference Optimization: this skips the separate reward model and optimizes the LLM directly on the preference pairs, which is cheaper and simpler, and now common. Constitutional AI, from Anthropic, in Bai and colleagues 2022: the model critiques and revises its own answers against a written set of principles, a constitution, generating much of its own alignment signal and reducing the need for human labels.

Fourth, domain fine-tuning by enterprises. On top of an already-aligned model, an enterprise may fine-tune, often with LoRA or PEFT, to specialize style, format, or narrow-domain behavior.

The key distinction — knowledge versus behavior. Fine-tuning changes behavior and style, not reliably knowledge. It's excellent for "always output this JSON shape," "adopt our tone," or "classify into our twelve categories." It is a poor and risky way to inject facts — facts learned this way can't be updated without retraining, may be hallucinated confidently, and can't be permissioned per-user.

To give a model current, factual, permissioned knowledge, use retrieval, not fine-tuning. This is one of the most important practical rules in the whole series.

When to use it (and when not to). Prompt first, then add retrieval, then fine-tune only when you need consistent behavior, format, or latency that prompting can't achieve, and you have quality training data and an evaluation harness.

Where it goes wrong. "The model doesn't know our products, let's fine-tune it on our catalog." Usually wrong — the catalog changes, and fine-tuning bakes in a stale snapshot. Retrieve the catalog instead.

A concrete example. A bank fine-tunes a model to always refuse to give individualized investment advice and to emit disclosures in a fixed format, which is behavior — but retrieves the client's actual portfolio at query time, which is knowledge.

Inference parameters: temperature, sampling, logprobs

Why it matters. At generation time you control how the model turns its next-token probability distribution into actual text. These knobs trade determinism and reliability against creativity and diversity. For enterprise systems — especially agents and structured outputs — you usually want the reliable end.

How it works — the main knobs. Temperature scales the probability distribution before sampling. Low, from zero to zero point three: the model almost always picks the highest-probability token, giving focused, repeatable, boring-but-correct output. High, from zero point eight to one point two: a flatter distribution, giving more diverse and creative output, but more error-prone and less consistent. For extraction, classification, tool-calling, and most agent work, use low temperature. For brainstorming or marketing copy, higher.

Top-p, also called nucleus sampling, samples only from the smallest set of tokens whose cumulative probability is at least p — for example, zero point nine. It caps the long tail of unlikely tokens.

Top-k samples only from the k most likely tokens.

Max tokens and stop sequences cap output length and define where to stop.

Frequency and presence penalties discourage repetition.

Seed, where supported, improves reproducibility but does not guarantee bit-identical output across infrastructure.

Logprobs: the model can return the log-probability of chosen and alternative tokens. This is quietly one of the most useful enterprise features because it gives you a confidence signal. Uses: flag low-confidence answers for human review, build classifiers or routers, detect when the model is unsure, and power certain evaluation methods.

How the knobs combine, so you know what to actually set. They're applied in sequence: temperature first reshapes the distribution, where flatter means more random, then top-p and top-k truncate its tail. You rarely tune all three at once. A practical default for reliable enterprise tasks: temperature at roughly zero and leave top-p and top-k at their defaults; reach for higher temperature, and maybe top-p around zero point nine, only for genuinely creative generation.

When to use it (and when not to). Default to low temperature for anything that must be correct or parseable. Raise it only for genuinely generative tasks. For structured outputs and tool calls, combine low temperature with schema enforcement.

Where it goes wrong. Determinism is not guaranteed even at temperature zero — floating-point nondeterminism, batching, and MoE routing can produce small variations. Don't build systems that assume

bit-exact reproducibility.

High temperature in agents amplifies compounding errors over a multi-step loop.

Confusing a confident tone with being correct. Logprobs measure the model's confidence in its own token choice, not factual accuracy — a fluent hallucination can have high logprobs.

A concrete example. A quote-generation agent runs at temperature zero for extracting line items and computing configurations, which must be exact, and uses logprobs to route any extraction below a confidence threshold to a human for review before the quote is written back to the system of record.

Why hallucinations occur

Why it matters. A hallucination is a fluent, confident output that is factually wrong or fabricated — an invented citation, a made-up API parameter, a plausible but false contract term. It's the defining risk of enterprise LLM deployment. Understanding why it happens tells you how to reduce it; you rarely eliminate it.

How it works — the root causes. First, the objective is plausibility, not truth. The model is trained to produce likely text. A confident, well-formed falsehood is often more likely than "I don't know." Nothing in the base objective rewards truth as such.

Second, knowledge is lossy and interpolated. Facts are compressed into weights; rare or fine-grained facts, such as a specific customer's data or a niche API signature, are poorly stored and get interpolated — the model fills gaps with plausible patterns.

Third, there is no inherent notion of "I don't know." Alignment can teach hedging, but the base model has no calibrated uncertainty about facts; it will complete the pattern.

Fourth, stale or absent knowledge. Anything after the training cutoff, or private to your enterprise, simply isn't in the weights.

Fifth, prompt and context pressure. Leading questions, contradictory context, or "lost in the middle" retrieval push the model to confabulate.

Sixth, reasoning errors. Multi-step problems accumulate mistakes; the model may state a wrong intermediate result confidently.

Seventh, evaluation incentives reward guessing. A widely-cited 2025 analysis — OpenAI's "Why Language Models Hallucinate," later in *Nature* in 2026 — argues models bluff partly because most benchmarks score "I don't know" the same as a wrong answer, which is zero, and reward a lucky confident guess. So training

and model selection favor confident guessing over calibrated abstention. The implied fix is to reward calibrated uncertainty in evaluations, not just add hallucination tests.

The mitigations, each expanded in later modules. Retrieval: ground answers in retrieved source text and require citations. This is the single biggest lever for factual tasks.

Tool calling: let the model compute — with a calculator, SQL, or a system-of-record query — instead of recalling.

Structured outputs plus validation: constrain the shape and check it.

Lower temperature, and confidence gating via logprobs.

Prompt techniques: "answer only from the provided context; if it's not there, say you don't know."

Verification layers and LLM-as-judge, plus human-in-the-loop for high-stakes outputs.

Reasoning models reduce reasoning and arithmetic errors but do not eliminate factual fabrication — and on summarization and faithfulness tasks, extended thinking can increase hallucination, because the long internal chain drifts from the source. Match mode to task: reasoning for math, logic, and code; base models for grounded summarization.

When to use it (and when not to). Match the mitigation stack to the stakes. A brainstorming assistant tolerates hallucination; an agent that writes a price into a customer quote needs retrieval grounding, tool-based computation, schema validation, and human approval.

Failure modes to name for the video series. Fabricated citations, URLs, or case law — a notorious legal example.

Confident wrong arithmetic.

Sycophancy — agreeing with an incorrect premise the user stated.

Context override — ignoring provided source text in favor of parametric memory.

Ungrounded extrapolation — inventing product features or SLA terms that sound right.

A concrete example. Ask a bare model "What's ACME Corp's contract renewal date?" and it may confidently invent one. The fix is architectural, not a better prompt: the agent must retrieve the record from the CRM system of record and answer only from it, citing the record — and say "I don't have that" when retrieval returns nothing. This is the bridge into the capability-extension and retrieval modules.

Module 1 takeaways

An LLM is a next-token predictor: fluent reasoning, unreliable knowledge.

Tokens drive cost and limits; embeddings encode meaning as geometry and power semantic search.

The transformer and attention make it all scale; attention's cost, which grows with the square of the sequence length, is why long context is expensive and retrieval beats stuffing.

The context window is finite working memory; bigger is not the same as better-used.

Training stages separate behavior, which is fine-tunable, from knowledge, which you should retrieve rather than fine-tune.

Inference knobs trade determinism for creativity; default to low temperature for enterprise; logprobs give a confidence signal.

Hallucination is intrinsic to the objective; you manage it with grounding, tools, structure, and human oversight — the subject of the rest of this series.